



Школа информатики,
физики и технологий

Прикладная математика и
информатика

Санкт-Петербург, 2025

Улучшенная нижняя оценка для константы Чватала-Санкоффа методом динамического программирования с окнами

Юкачев Дмитрий Евгеньевич, студент 4 курс

Введение в предметную область

Longest Common Subsequence (LCS) — наибольшая общая подпоследовательность двух строк. Подпоследовательность получается удалением некоторых элементов без изменения порядка оставшихся.

Случайные бинарные строки — две независимые строки X, Y длины n , где каждый элемент независимо выбирается из $\{0, 1\}$ с вероятностью $1/2$.

Проблема Чватала–Санкоффа (1975): Для двух случайных бинарных строк длины n , матожидание длины их LCS асимптотически линейно:

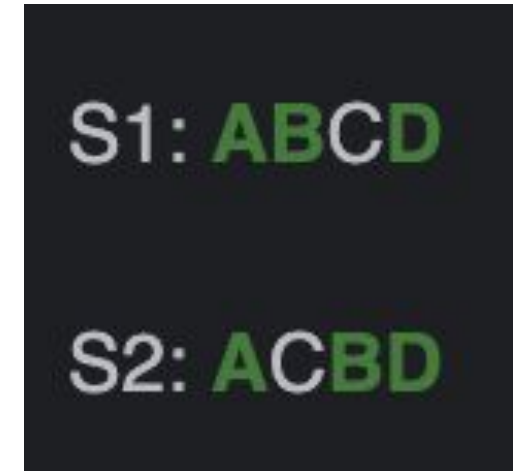
$$E[L_n] \sim \gamma_2 \cdot n \text{ при } n \rightarrow \infty$$

где γ_2 — константа Чватала–Санкоффа для бинарного алфавита.

Открытая проблема: Точное значение γ_2 неизвестно с 1975 года!

Текущие границы:

- Нижняя: $\gamma_2 \geq 0.792666$ (Heineman et al., 2024)
- Верхняя: $\gamma_2 \leq 0.826280$ (Lueker, 2009)
- Ожидаемая $\gamma_2 \sim 0.811$ (Метод Монте-Карло)



Пример:

$LCS("ABCD", "ACBD") = "ABD"$
(длина 3)



Цели и задачи исследования

Цель работы:

Получить улучшенную строгую нижнюю оценку для константы Чватала–Санкоффа γ_2 , превосходящую современные результаты.

Задачи:

1. Разработать вычислительный метод для оценки $E[LCS]$ случайных бинарных строк
2. Эффективно реализовать данный алгоритм
3. Провести масштабные численные эксперименты для получения нижней оценки
4. Сравнить полученные результаты с лучшими известными оценками из литературы
5. Оптимизировать реализацию для работы с большими параметрами (N , $shift$, K)



Результаты

Текущий результат:

Получена новая лучшая строгая нижняя оценка для γ_2 :
 $\gamma_2 \geq 0.7959$

при параметрах: $N=12$, $MAX_SHIFT=25$, $K=10,000$

Параметры эксперимента:

- Длина окна: $N = 12$ бит
- Максимальный сдвиг: $SH = 25$
- Длина строк: $K = 10,000$
- Время вычислений: ~ 6 часов

Авторы	Год	Нижняя оценка γ_2
Chvátal & Sankoff	1975	≥ 0.77
Dančák & Paterson	1995	≥ 0.773911
Lueker	2003	≥ 0.788071
Heineman et al.	2024	≥ 0.792666
Данная работа	2025	≥ 0.7959



Предпосылки решения

Основная идея:

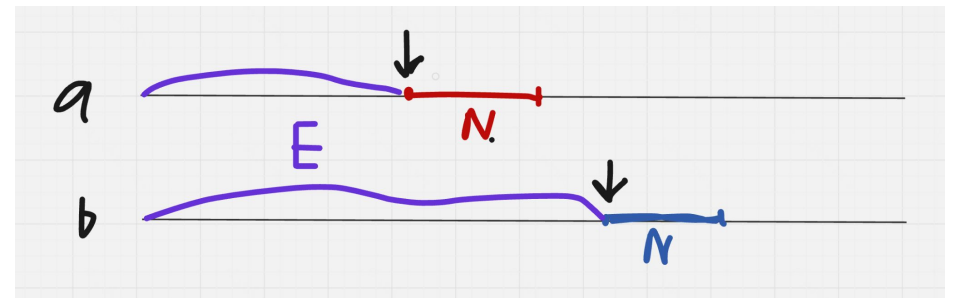
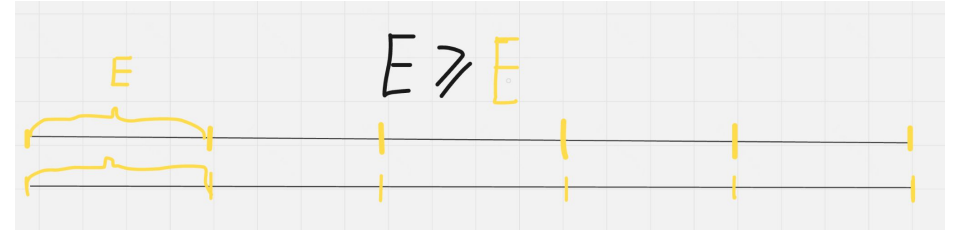
Для получения нижней оценки можно считать матожидание LCP для `кусочков` строк

Примеры, полученные полным перебором:

- Для кусков длины 1, $E=0.5$
- Для кусков длины 2, $E=0.5625$
- Для кусков длины 10, $E=$

Полный перебор работает за $O(4^n \cdot n^2)$, для больших кусков нужно придумать что-то еще...

Давайте генерировать строчки на ходу, смотря на N символов вперед





Описание метода: Динамическое программирование с окнами

Основная идея:

Вместо анализа всех возможных LCS целиком, рассматриваем локальные решения с "окнами" обзора на N символов вперёд.

Состояние DP:

$\text{solve}(L, \text{shift}, \text{maskA}, \text{maskB})$

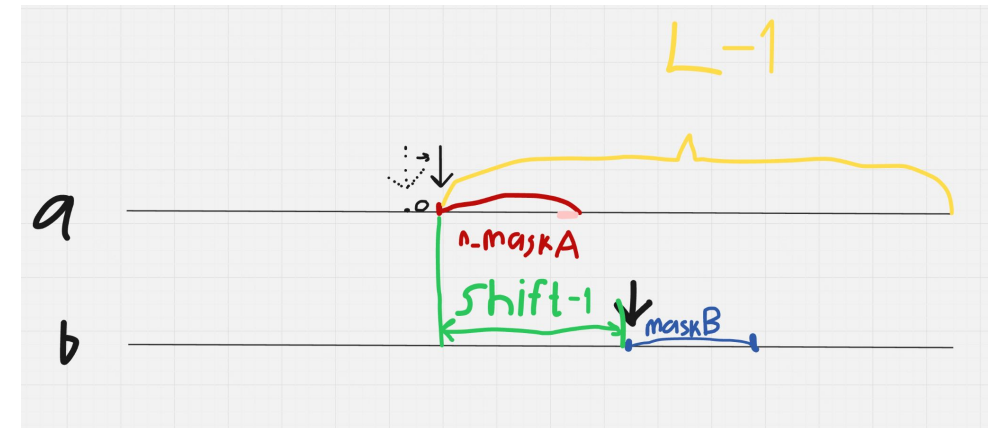
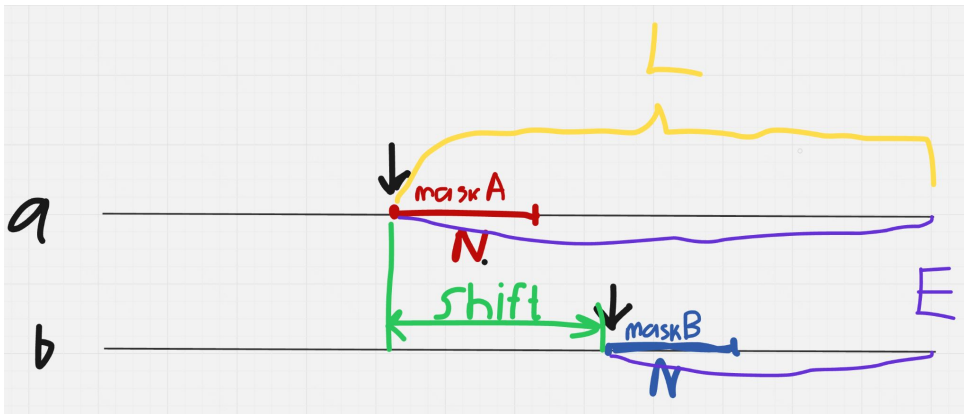
- L — оставшаяся длина строк
- shift — текущий сдвиг $|i - j|$ между позициями в строках A и B
- $\text{maskA}, \text{maskB}$ — битовые маски следующих N символов строк A и B

Возвращает: $E[\text{LCS}]$ для данной конфигурации

Время работы/память: $O(K \cdot \text{MAX_SH} \cdot 4^N)$

Что делаем	Условие	L	Shift
Двигаем вперед a	$\text{shift} > 0$	$-- 1$	$-- 1$
Меняем a и b	$\text{shift} == 0$	const	$= 1$
Двигаем вперед b	$\text{shift} < \text{MAX_SH}$	const	$+= 1$
Также двигаем и a	$\text{shift} == \text{MAX_SH}$	$-- 1$	const
Берём символ в LCS	$\text{maskA}[0] == \text{maskB}[0]$	$-- 1$	const

Таблица переходов динамики





Оптимизации реализации

Вычислительные оптимизации:

1. Итеративный подход:

- Замена рекурсии на итерацию по L от 1 до K
- Избегание overhead вызовов функций

2. Оптимизации компилятора:

- `#pragma GCC optimize("Ofast,no-stack-protector,unroll-loops,fast-math")`
- `#pragma GCC target("tune=native")`
- `#pragma GCC target("avx2")`

4. Выбор параметров:

- N=12: размер окна
- MAX_SH=25: максимальный сдвиг
- K=10'000: длина строк
- Время работы 6 часов
- Результат 0.7959

Сложность:

- Память: $O(\text{MAX_SH} \cdot 4^N)$
- Время: $O(K \cdot \text{MAX_SH} \cdot 4^N)$

```
int main() {
    // dp[L][sh][maskA][maskB] -> dp[sh][maskA][maskB]
    for (int l = 1; l <= L; l++) {
        memcpy(dp_one_sh, dp[l], n * sizeof(dp_one_sh));
        for (int sh = MAX_SH; sh >= 0; sh--) {
            memcpy(dp_temp, dp[sh], n * sizeof(dp_temp));
            if (sh < MAX_SH) {
                memcpy(dp_plus_one_sh, dp[sh+1], n * sizeof(dp_plus_one_sh));
            }
            for (int maskA = 0; maskA < (1<<N); maskA++) {
                for (int maskB = 0; maskB < (1<<N); maskB++) {
                    float res = 0;
                    for (int nextA = 0; nextA < 2; nextA++) {
                        for (int nextB = 0; nextB < 2; nextB++) {
                            int nextMaskA = (maskA >> 1) | (nextA << (N-1));
                            int nextMaskB = (maskB >> 1) | (nextB << (N-1));
                            float tmp_res = 0;
                            if ((maskA & 1) == (maskB & 1)) {
                                tmp_res = 1 + dp_temp[nextMaskA][nextMaskB];
                            } else {
                                if (sh > 0) {
                                    tmp_res = dp[sh-1][nextMaskA][maskB];
                                } else {
                                    tmp_res = dp_one_sh[maskB][nextMaskA];
                                }
                                if (sh < MAX_SH) {
                                    tmp_res = max(a: tmp_res, b: dp_plus_one_sh[maskA][nextMaskB]);
                                } else {
                                    tmp_res = max(a: tmp_res, b: dp_temp[nextMaskA][nextMaskB]);
                                }
                            }
                            res += tmp_res;
                        }
                    }
                    dp[sh][maskA][maskB] = res;
                }
            }
        }
    }
}
```



Направления дальнейших исследований

Возможные улучшения:

1. Алгоритмические оптимизации:

- Отсечение бесполезных масок (жадная стратегия при LCS (видимая часть) $\geq 0.5 \cdot N$)
- Потенциал: ускорение в 100 раз

2. Вычислительные улучшения:

- Масштабирование на кластер/суперкомпьютер
- Добавление паралелизации

3. Увеличение параметров:

- $N=12$, $SH=50$, $K=1,000,000 \rightarrow$ ожидаемая оценка $\sim 0.8+$

4. Другие оценки по теме:

- Оценка сверху
- Оценки снизу для алфавитов больших размеров



Основные источники и публикации

Базовые работы:

- Chvátal, V., Sankoff, D. (1975). Longest common subsequences of two random sequences. J. Appl. Probab. 12(2), 306–315.
- Lueker, G. S. (2009). Improved bounds on the average length of longest common subsequences. J. ACM 56(3), 17:1–17:38.
- Heineman, G. T., Miller, C., Reichman, D., Salls, A., Sarkozy, G., Soiffer, D. (2024). Improved Lower Bounds on the Expected Length of Longest Common Subsequences. ArXiv:2407.10925
- Tiskin, A. (2023). Проблема Хватала–Санкоффа как задача символической динамики. Зап. научн. сем. ПОМИ 528, 214–237.

Данная работа:

Название: Улучшенная нижняя оценка для константы Чватала-Санкоффа методом динамического программирования с окнами

Автор: Юкачев Д. Е.

Год: 2025

Результат: $\gamma_2 \geq 0.7959$

